JAVA INTERVIEW PREPARATION

CHAPTER 4

JAVA GENERICS:
TYPE SAFETY, VARIANCE, AND ERASURE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Java Generics: Type Safety, Variance, and Type Erasure

Senior / Lead Java Developer Reading Chapter

Generics allow Java APIs to express relationships between types and move many errors from runtime to compile time. A `List<Customer>` communicates more than a raw `List`: callers know what can be inserted, what will be returned, and which operations are safe without casting.

The syntax is often introduced through collections, but senior-level use requires a deeper model. Java generics are invariant, wildcards describe controlled variance, type parameters represent relationships, and type erasure limits what the runtime knows. Poorly designed generic APIs can be technically type-safe while remaining difficult to understand, or can hide heap pollution until a distant `ClassCastException` occurs.

From Raw Types to Compile-Time Safety

Before generics, collections stored `Object`. The compiler could not verify which values belonged in them.

```
List values = new ArrayList();
values.add("customer-100");
values.add(42);

String customer = (String) values.get(1); // ClassCastException
```

The error is introduced when the integer is added, but it becomes visible later when a caller performs a cast. The stack trace points to the symptom rather than the original mistake.

A parameterized collection makes the intended element type part of the contract:

```
List<String> values = new ArrayList<>();
values.add("customer-100");
values.add(42); // compile-time error

String customer = values.get(0); // no cast required
```

Generics do not merely remove casts. They establish relationships. If a method accepts `List<Order>` and returns `Optional<Order>`, the compiler can trace the element type through the API.

Raw types remain in Java for compatibility with older code. Using a raw type disables part of the generic contract and should normally produce a warning. `List<?>` is different: it means a List of one specific but unknown type and preserves type safety.

Generic Classes and Meaningful Type Parameters

A generic class introduces a type parameter that its methods and state can use consistently.

```
public final class Result<T> {

    private final T value;
    private final String error;

    private Result(T value, String error) {
        this.value = value;
        this.error = error;
    }

    public static <T> Result<T> success(T value) {
        return new Result<>(Objects.requireNonNull(value), null);
    }

    public static <T> Result<T> failure(String error) {
        return new Result<>(null, Objects.requireNonNull(error));
    }

    public Optional<T> value() {
        return Optional.ofNullable(value);
    }
}
```

`T` represents the successful value type. A `Result<Customer>` and `Result<Order>` share implementation but preserve different compile-time contracts.

The static factory methods declare their own `<T>` before the return type because static methods cannot use the class instance's type parameter directly. Type inference often allows the caller to omit explicit arguments:

```
Result<Customer> result = Result.success(customer);
```

Type-parameter names should communicate their role. Single letters such as `T`, `E`, `K`, and `V` are conventional for short generic structures. In APIs with several concepts, names such as `REQUEST`, `RESPONSE`, or `ENTITY` may make relationships clearer.

Generics should not be added when there is no meaningful relationship to preserve. A class with many unrelated parameters can become harder to use than several focused abstractions.

Why Java Generics Are Invariant

Although `Integer` is a subtype of `Number`, `List<Integer>` is not a subtype of `List<Number>`.

```
List<Integer> integers = new ArrayList<>();
List<Number> numbers = integers; // compile-time error
```

If that assignment were legal, a caller could corrupt the Integer list:

```
numbers.add(3.14); // valid for List<Number>

Integer value = integers.get(0); // would no longer be safe
```

The compiler rejects the assignment to protect the element-type guarantee.

```
Integer extends Number

does not imply

List<Integer> extends List<Number>

because List<Number> permits adding any Number
```

Arrays behave differently: they are covariant.

```
Integer[] integers = new Integer[10];
Number[] numbers = integers;

numbers[0] = 3.14; // ArrayStoreException at runtime
```

The array remembers its runtime component type and performs a store check. Generics choose compile-time safety instead, avoiding the delayed runtime failure.

Wildcards Describe Variance at an API Boundary

Wildcards do not make a generic type globally covariant or contravariant. They describe what a particular reference is allowed to do.

`List<? extends Number>` means a List whose element type is some specific unknown subtype of Number.

```
double total(List<? extends Number> values) {
    double result = 0;

    for (Number value : values) {
        result += value.doubleValue();
    }

    return result;
}
```

The method can receive `List<Integer>`, `List<Long>`, or `List<BigDecimal>` because reading each element as Number is safe.

It cannot safely add an Integer:

```
values.add(1); // compile-time error
```

The actual list might be `List<Double>`. The compiler knows there is a subtype, but not which subtype. The only value universally safe to add is `null`, which is rarely useful.

`List<? super Integer>` means a List of Integer or one of its supertypes.

```
void addDefaults(List<? super Integer> destination) {
    destination.add(10);
    destination.add(20);
}
```

The method can receive `List<Integer>`, `List<Number>`, or `List<Object>`. Adding an Integer is safe in all cases. Reading produces `Object` because the compiler does not know the list's more specific declared type.

PECS: Producer Extends, Consumer Super

PECS is a memory aid for choosing wildcard direction:

- If a structure produces values for the method, use `? extends T`.
- If it consumes values supplied by the method, use `? super T`.

The standard copy relationship is:

```
static <T> void copy(
        List<? extends T> source,
        List<? super T> destination) {

    for (T value : source) {
        destination.add(value);
    }
}
```

```
List<Integer> source
         |
         | produces T values
         v
      copy()
         |
         | consumes T values
         v
List<Number> destination
```

PECS is about the role of the collection in that method, not a permanent property of the object. The same list can be a producer in one API and a consumer in another.

When a parameter both produces and consumes exactly the same type, use `List<T>` rather than a wildcard. Avoid wildcard return types when callers need to name or mutate the returned value; returning `List<? extends Animal>` often transfers uncertainty to every caller.

Unbounded Wildcards and Unknown Types

`List<?>` means a List of an unknown element type while preserving the fact that one consistent element type exists.

```
void printSize(List<?> values) {
    System.out.println(values.size());
}
```

It is safer than raw `List` because arbitrary values cannot be added.

```
void unsafe(List values) {
    values.add(new Customer()); // unchecked operation
}

void safeUnknown(List<?> values) {
    values.add(new Customer()); // compile-time error
}
```

Use an unbounded wildcard when the operation does not depend on the element type, such as checking size, clearing the collection, or formatting elements through `Object` behavior.

`List<Object>` is not a replacement. It specifically means a list that may contain any Object and can accept new Objects. A `List<String>` cannot be passed as `List<Object>` because generics are invariant, but it can be passed as `List<?>`.

Generic Methods and Type Inference

A generic method declares a relationship local to one operation.

```
static <T> T firstOrDefault(
        List<? extends T> values,
        T defaultValue) {

    return values.isEmpty() ? defaultValue : values.get(0);
}
```

The compiler infers `T` from all arguments and the target type.

```
Number number = firstOrDefault(List.of(1, 2), 0.5);
```

Here the compiler finds a common type that can represent Integer elements and the Double default.

Inference can produce a broader type than expected when arguments have different hierarchies. If an API becomes hard to call without explicit type witnesses, the relationship may be too complicated.

```
Customer customer = RepositoryUtil.<Customer>requireFound(optional);
```

Explicit type arguments are valid, but ordinarily inference should keep the call readable.

Bounded Type Parameters

An upper bound allows generic code to use operations from a required contract.

```
static <T extends Comparable<? super T>> T maximum(
        List<? extends T> values) {

    if (values.isEmpty()) {
        throw new IllegalArgumentException("Values are required");
    }

    T maximum = values.get(0);

    for (T value : values) {
        if (value.compareTo(maximum) > 0) {
            maximum = value;
        }
    }

    return maximum;
}
```

`T extends Comparable<? super T>` means T can be compared through a comparator contract defined for T or one of T's supertypes. This is more flexible than `Comparable<T>` for inherited comparison contracts.

Multiple bounds are possible:

```
<T extends Auditable & Comparable<? super T>>
```

If a class bound exists, it must appear first, followed by interface bounds. Multiple bounds should represent a genuine algorithm requirement rather than an attempt to encode an entire application architecture into one signature.

Wildcard Capture

Sometimes an API receives `List<?>`, and a helper method must treat its unknown element type consistently.

```
static void reverseUnknown(List<?> values) {
    reverseCaptured(values);
}

private static <T> void reverseCaptured(List<T> values) {
    for (int left = 0, right = values.size() - 1;
         left < right;
         left++, right--) {

        T temporary = values.get(left);
        values.set(left, values.get(right));
        values.set(right, temporary);
    }
}
```

The helper captures the wildcard as one type `T`. The method still does not know what T is, but it knows every read and write uses that same type.

Wildcard-capture errors often indicate that a private generic helper is needed. They can also indicate an overly vague public API. Do not expose complicated capture mechanics to ordinary callers when a clearer type parameter can express the relationship.

Type Erasure

Java implemented generics with type erasure to preserve compatibility with pre-generics bytecode and the existing JVM model.

At compilation, type parameters are checked and then largely erased to their bounds. Casts and bridge methods are inserted where necessary.

Conceptually:

```
public T get() {
    return value;
}
```

may erase toward:

```
public Object get() {
    return value;
}
```

The caller's compiled code performs the type-specific cast required by the generic contract.

As a result, these objects normally have the same runtime class:

```
List<String> strings = new ArrayList<>();
List<Integer> integers = new ArrayList<>();

strings.getClass() == integers.getClass(); // true
```

The JVM does not create a separate `ArrayList<String>` class and `ArrayList<Integer>` class.

Generic signature metadata can remain in class files and be inspected by reflection for declarations such as fields and method parameters. That does not mean every runtime object carries its complete generic arguments. Reflection code must distinguish declared generic metadata from the erased runtime class.

Restrictions Caused by Erasure

Because T is not generally a reified runtime class, Java rejects several operations.

```
new T();                // cannot directly construct T
T.class;                // no class literal for T
new T[10];              // cannot create a generic array
value instanceof T;     // cannot test an erased type parameter
```

When construction is required, accept a factory or type token:

```
public final class Factory<T> {

    private final Supplier<? extends T> supplier;

    public Factory(Supplier<? extends T> supplier) {
        this.supplier = supplier;
    }

    public T create() {
        return supplier.get();
    }
}
```

```
Factory<ArrayList<String>> factory =
    new Factory<>(ArrayList::new);
```

For reflection or serialization, a `Class<T>` works for non-parameterized types:

```
T decode(String json, Class<T> type)
```

It cannot fully represent `List<Customer>` because `List.class` loses the element type. Frameworks use richer type tokens such as a parameterized type reference, often captured through an anonymous subclass.

Bridge Methods

Erasure can make an overriding method's erased signature differ from its parent. The compiler may generate a synthetic bridge method to preserve polymorphism.

```
interface Converter<T> {
    T convert(String value);
}

final class IntegerConverter implements Converter<Integer> {

    @Override
    public Integer convert(String value) {
        return Integer.valueOf(value);
    }
}
```

After erasure, the interface method returns Object. At the bytecode level, where the method descriptor includes the return type, the compiler can generate a synthetic bridge conceptually equivalent to the following. This is an explanation of generated bytecode, not a second method that could legally be declared beside the source method using only a different return type.

```
// Conceptual synthetic bridge generated by the compiler
public Object convert(String value) {
    return convert(value); // delegates to the Integer-returning method
}
```

Bridge methods normally remain invisible to application design. They can appear in reflection, stack traces, coverage reports, or bytecode inspection. Understanding them prevents treating a compiler-generated method as duplicate business logic.

Heap Pollution

Heap pollution occurs when a variable of a parameterized type refers to an object that does not satisfy that parameterized contract.

Raw types can introduce it:

```
List<String> strings = new ArrayList<>();
List raw = strings;

raw.add(42); // unchecked warning

String value = strings.get(0); // ClassCastException
```

The heap contains an Integer inside an object viewed as `List<String>`. The compiler warning identifies the unsafe boundary. Suppressing the warning without proving safety moves the failure elsewhere.

Unchecked casts are sometimes necessary at framework boundaries, but they should be narrow, documented, and immediately validated where possible.

```
@SuppressWarnings("unchecked")
private static <T> T trustedFrameworkValue(Object value) {
    return (T) value;
}
```

This method is safe only if an external invariant guarantees the value's type. The annotation does not make the cast safe; it merely hides the warning.

Generic Varargs and `@SafeVarargs`

Varargs are implemented with arrays, while generic arrays cannot safely represent parameterized element types. Combining them can create heap pollution.

```
static void unsafe(List<String>... groups) {
    Object[] array = groups;
    array[0] = List.of(42);

    String value = groups[0].get(0); // ClassCastException
}
```

The compiler warns because the runtime array cannot fully enforce `List<String>` as its component type.

`@SafeVarargs` is a programmer assertion that the method does not perform unsafe operations on the varargs array and does not expose it to code that might do so.

```
@SafeVarargs
static <T> List<T> combine(List<? extends T>... groups) {
    List<T> result = new ArrayList<>();

    for (List<? extends T> group : groups) {
        result.addAll(group);
    }

    return result;
}
```

The annotation must be justified by implementation. Do not add it merely to silence a warning. A collection parameter may provide a cleaner API than generic varargs.

Designing Usable Generic APIs

The best generic signature exposes useful flexibility without forcing callers to solve a type puzzle.

Prefer interface types and wildcard inputs when callers reasonably use subtypes:

```
public <T> List<T> transform(
        Collection<? extends T> source,
        UnaryOperator<T> operation) {
    // implementation
}
```

But do not add wildcards mechanically. If the operation fundamentally requires one exact type, `List<T>` communicates that relationship more clearly.

Prefer type parameters when the same type appears in multiple positions and the relationship matters:

```
static <T> T choose(T first, T second)
```

Prefer a wildcard when only flexible consumption or production matters:

```
static void notifyAll(Collection<? extends Event> events)
```

Avoid returning raw types, exposing unchecked casts, or returning wildcard-heavy structures that callers cannot use. Generic complexity is an API cost. A small domain-specific type can be clearer than a signature with many nested wildcards.

Production Perspective

Generic failures often surface far from the code that introduced them. A raw deserialization layer inserts the wrong type into a collection, and a scheduled job fails hours later during iteration. A suppressed cast in a cache adapter works for most keys but fails for one configuration. A framework loses nested generic information and deserializes objects as maps. A generic varargs method pollutes an array and causes a cast failure in another component.

When diagnosing these problems, follow the value back to the unchecked boundary:

- Compile with unchecked warnings enabled and treat new warnings seriously.
- Inspect raw types, unchecked casts, reflection, serialization, and varargs boundaries.
- Capture the runtime class of the unexpected value safely in diagnostics.
- Verify the declared generic metadata supplied to frameworks.
- Check whether a bridge method or proxy affects reflective method selection.
- Add tests that cross the boundary with more than one type argument.

The `ClassCastException` line is often only where the compiler-inserted cast finally detects earlier heap pollution.

Interview-Ready Summary

Generics provide compile-time type safety and express relationships between inputs, outputs, and stored values. Java generics are invariant: even when Integer extends Number, `List<Integer>` is not a subtype of `List<Number>` because that would allow unsafe insertion.

Wildcards provide use-site variance. `? extends T` is suitable for a producer that the method reads from, while `? super T` is suitable for a consumer the method writes to. PECS is a useful guide, but the correct signature follows the role of the value in that operation.

Type parameters should be used when the same unknown type connects several positions. Bounds allow generic algorithms to require capabilities such as Comparable. Wildcard capture lets a helper treat one unknown type consistently.

Java implements generics primarily through type erasure. Parameterizations generally share the same runtime class, direct `new T()`, `T.class`, generic arrays, and `instanceof T` are unavailable, and the compiler may insert casts and bridge methods.

Raw types, unchecked casts, and generic varargs can introduce heap pollution. Warnings should be isolated and justified rather than broadly suppressed. A senior generic API balances flexibility with readability and prevents uncertainty from spreading to callers.

Revision Notes

- Generics move many type errors from runtime to compile time.
- Raw types disable generic safety; `List<?>` preserves an unknown but consistent type.
- Java generics are invariant: `List<Integer>` is not a `List<Number>`.
- Arrays are covariant and may fail with `ArrayStoreException` at runtime.
- `? extends T` is read-oriented production; `? super T` is write-oriented consumption.
- PECS means Producer Extends, Consumer Super.
- Use `List<T>` when an operation both reads and writes the same exact type.
- Use a type parameter when one type relationship connects multiple positions.
- Bounds express capabilities required by a generic algorithm.
- Wildcard capture gives one unknown type a helper-method name.
- Erasure means parameterizations normally share one runtime class.
- Erasure prevents direct `new T()`, `T.class`, `new T[]`, and `instanceof T`.
- Use factories, suppliers, or type tokens when runtime construction or metadata is needed.
- Bridge methods preserve overriding behavior after erasure.
- Heap pollution means a parameterized reference points to incompatible contents.
- Treat raw-type and unchecked warnings as unsafe-boundary indicators.
- `@SafeVarargs` is a safety assertion, not a warning-suppression shortcut.
- Prefer a readable domain API over unnecessary nested generic complexity.